

Data Processing Assignment — Summary Report

Vabhravi Pandey | Pandas Data Cleaning & Delta Lake Incremental Processing (SCD1 / SCD2)

Part 1 — Pandas Data Exploration & Cleaning

Dataset: Superstore Dataset Final (Kaggle, vivek468). The raw file contained 10,800 rows across 21 columns, but the final 806 rows were not genuine order records — they were leftover data from other sheets in the original workbook (People / Returns tabs) that had been concatenated into the Orders CSV. These were identified via non-numeric **Row ID** values and dropped. Among the remaining 9,994 genuine order rows, 11 had a missing **Postal Code**, filled using the most common postal code per City. Data types were corrected (dates parsed as datetime), 0 exact duplicates were found, and two derived columns — **unit_price** and **total_amount** — were added and verified against the original Sales figures.

Metric	Value
Raw rows / columns	10,800 / 21
Corrupted rows dropped	806
Missing Postal Codes filled	11
Duplicate rows removed	0
Final cleaned rows / columns	9,994 / 23

Part 2 — Incremental Processing with Delta Lake

A synthetic customer dataset was used to simulate an incremental ETL scenario: 25 customers in a master table (with 2 duplicate rows and 3 missing phone numbers deliberately introduced), plus an incremental batch of 10 records — 5 updates to existing customers and 5 brand-new customers. Both datasets were loaded into genuine Delta Lake tables (real transaction log, Parquet files, ACID MERGE) using the **deltalake** (delta-rs) Python engine — a lightweight alternative to PySpark that needs no JVM or Spark cluster to set up. Equivalent PySpark SQL is documented alongside every MERGE in the notebook for reference.

Two incremental-load strategies were implemented and compared:

SCD Type 1 (overwrite): a single MERGE INTO ... WHEN MATCHED UPDATE / WHEN NOT MATCHED INSERT keeps exactly one row per customer, always reflecting the latest state. No history is retained.

SCD Type 2 (versioned history): a two-step MERGE pattern — first expiring the current row for any customer with an incoming change (`is_current = false`), then inserting a fresh current-version row for anything without an active match (catching both new customers and just-expired ones). This preserves a full, queryable audit trail of every change.

Metric	SCD1	SCD2
Total rows after merge	30	35
Current rows	30	30
Historical rows	0	5
Duplicate customer_ids	0	n/a (by design)

Validation: row counts and duplicate checks were asserted programmatically in the notebook (all checks passed), and the Delta transaction log for the SCD2 table shows the expected commit sequence — **v0: WRITE, v1: MERGE (expire), v2: MERGE (insert)** — confirming the incremental load ran as real, auditable Delta Lake operations rather than a full table rewrite.

Files

notebooks/01_pandas_data_cleaning.ipynb · notebooks/02_delta_lake_incremental.ipynb · data/ (raw + cleaned CSVs for both tasks) · screenshots/ (execution evidence by stage) · README.md